

Практическое задание № 8 Матрицы и динамическая память.

Оценка 4:

Выполнить задачу согласно варианту. Не использовать динамическое выделение памяти.

Оценка 5:

Выполнить задачу согласно варианту. Использовать динамическое выделение памяти (malloc, calloc, realloc, free). Использовать функции для подзадач.

1. Сформировать матрицу В путем вычеркивания строки и столбца матрицы А, на пересечении которых находится максимальный элемент матрицы А. Определить количество строк, в которых есть нули и записать в обратном порядке главную диагональ матрицы.

2. Упорядочить элементы каждой строки по возрастанию. Определить, есть ли в матрице нули, и найти среднее арифметическое элементов матриц. Поменять местами главную и побочную диагонали.

3. Транспонировать в матрице квадрат, в правом нижнем углу которого расположен максимальный элемент. Определить среднее арифметическое значение исходной матрицы и транспонированного квадрата отдельно. Заменить нули в матрице на скаляр, введенный с клавиатуры.

4. Получить матрицу $C[m,k]$ путем умножения матрицы $A[m,n]$ на матрицу $B[n,k]$ и определить в результирующей матрице строку с наибольшим количеством отрицательных элементов. Заменить в полученной матрице отрицательные элементы на их количество и записать в обратном порядке любую строку матрицы.

5. Элементы тех столбцов матрицы, в которых есть нули, увеличить на скаляр, равный количеству нулей в соответствующем столбце матрицы. Транспонировать полученную матрицу и поменять местами первый и последний столбец исходной матрицы.

6. Умножить в матрице квадрат, в правом верхнем углу которого расположен минимальный элемент, на число, вводимое с клавиатуры. Определить сумму элементов этого квадрата. Сформировать из этого квадрата массив.

7. Получить матрицу $D[m,n]$ путем умножения матрицы $B[m,k]$ на матрицу $E[k,n]$. Определить в результирующей матрице столбец с наибольшей суммой отрицательных элементов. Упорядочить этот столбец по убыванию.

8. Транспонировать в матрице квадрат, в правом верхнем углу которого расположен самый первый ноль. Определить, сколько всего нулей в исходной матрице, и, если их больше чем количество строк в матрице, заменить нули на это количество.

9. Упорядочить по возрастанию столбец с максимальным произведением положительных элементов. Сформировать массив из

сумм элементов главной и побочной диагоналей. Записать в обратном порядке элементы побочной диагонали.

10. Элементы тех строк матрицы, в которых есть отрицательные элементы увеличить на скаляр, равный размеру матрицы. Определить строку с максимальным количеством отрицательных элементов и упорядочить по возрастанию любой столбец матрицы.

11. Разделить в матрице квадрат, в левом верхнем углу которого расположен минимальный элемент, на число, равное сумме всех остальных элементов матрицы. Записать в обратном порядке главную диагональ матрицы.

12. Сформировать матрицу путем вычеркивания строки и столбца матрицы A , на пересечении которых находится минимальный элемент матрицы A . Определить количество нулей в сформированной матрице и поменять местами первую и последнюю строки матрицы.

13. Умножить в матрице квадрат, в левом нижнем углу которого расположен максимальный элемент, на число, равное количеству нулевых элементов в побочной диагонали. Заменить все отрицательные элементы матрицы на их модули.

14. Упорядочить элементы каждого столбца матрицы по убыванию и найти сумму положительных элементов в том столбце, где было больше всего перестановок при упорядочивании. Записать в обратном порядке главную диагональ матрицы

15. Элементы тех столбцов матрицы A , суммы которых больше 12, уменьшить на элемент главной диагонали соответствующего столбца. Определить количество таких строк. Побочную диагональ упорядочить по убыванию.

16. Элементы строк матрицы увеличить на средние значения положительных элементов своих строк. Опередить сумму всех элементов матрицы. Сформировать массив из элементов, расположенных выше главной и побочной диагоналей. Вычислить количество отрицательных элементов полученного массива.

17. Упорядочить элементы каждого столбца матрицы по возрастанию. Найти количество положительных элементов в том столбце, где было меньше всего перестановок при упорядочивании, а затем, заменить элементы этого столбца на нули.

18. Сформировать массив B из элементов расположенных выше главной и побочной диагоналей матрицы A . Определить, сколько элементов этого массива меньше среднего значения элементов исходной матрицы. Упорядочить главную диагональ матрицы.

19. Сформировать массив C из элементов матрицы A , расположенных ниже главной и побочной диагоналей и определить, сколько элементов этого массива меньше числа, равного сумме минимальных положительных элементов строк матрицы. Упорядочить побочную диагональ матрицы.

20. Сформировать массив B из максимальных элементов строк исходной матрицы A . Получить матрицу C умножением исходной

матрицы A на вектор B . Определить, есть ли в матрице A нули и если есть, то записать в обратном порядке элементы полученного массива.

21. Элементы тех строк матрицы, суммы которых меньше 15, увеличить на диагональные элементы побочной диагонали соответствующих строк. Определить строку с наибольшим количеством отрицательных элементов .

22. Сформировать массив B из попарной суммы элементов главной и побочной диагоналей исходной матрицы. Определить, сколько элементов этого массива меньше максимального значения заданной матрицы и если таких элементов нет, то упорядочить по убыванию сформированный массив.

23. Исходную матрицу упорядочить в порядке убывания сумм ее строк. Определить среднюю сумму строк, вычислить количество строк, имеющих сумму меньше средней, и вывести на печать исходную и упорядоченную матрицы.

24. Сформировать массив C из элементов, расположенных правее главной и побочной диагоналей, и вычислить сумму тех элементов этого массива, которые меньше числа, расположенного на пересечении диагоналей исходной матрицы.

25. Упорядочить все элементы матрицы в порядке убывания и определить сумму положительных элементов правой нижней четверти исходной матрицы. Определить, есть ли вне этого квадрата нули.

26. Сформировать массив В из элементов матрицы А, расположенных левее главной и побочной диагоналей, и вычислить произведение тех элементов этого массива, которые больше числа, расположенного на пересечении диагоналей исходной матрицы. Найти минимум в исходной матрице.

27. Упорядочить все элементы матрицы в порядке возрастания и определить количество положительных элементов левой верхней четверти исходной матрицы. Определить сумму положительных элементов, расположенных под побочной диагональю, и вывести этот треугольник.

28. Сформировать массив D, содержащий суммы отрицательных элементов каждого столбца, все элементы полученного массива возвести в квадрат и определить столбец, в котором отрицательных элементов в исходной матрице больше. Вывести на печать элементы, расположенные ниже главной и побочной диагоналей.

29. Поменять в матрице правый верхний и левый нижний треугольники матрицы. Определить, сколько значений полученной матрицы совпадает с соответствующим значением исходной матрицы и вычислить минимальный положительный элемент в главной диагонали.

30. Упорядочить элементы каждой строки по возрастанию и найти среднее арифметическое элементов того столбца, где разместился минимальный элемент матрицы и вывести нижний левый треугольник матрицы.

Под Звездочкой (+ балл):

Реализовать простой кэш для **целых чисел** с алгоритмом вытеснения Least Recently Used (LRU) используя динамические массивы. Кэш должен хранить ограниченное количество элементов, а при превышении лимита - вытеснять самый давно неиспользуемый элемент.

Алгоритм работы:

- При запросе значения сначала проверять его наличие в кэше
- Если значение найдено - обновить время его использования (**время = счетчик обращений к кэшу**)
- Если не найдено - добавить в кэш:
 - При наличии свободного места - добавить в первую свободную ячейку
 - При заполнении кэша - найти элемент с самым старым временем использования и заменить его

Пример работы:

```
Запрос: 1 -> НЕ найдено, добавляем в кэш
Кэш (1/3): [ 1(1)  пусто  пусто ]

Запрос: 2 -> НЕ найдено, добавляем в кэш
Кэш (2/3): [ 1(1)  2(2)  пусто ]

Запрос: 3 -> НЕ найдено, добавляем в кэш
Кэш (3/3): [ 1(1)  2(2)  3(3) ]

Запрос: 1 -> НАЙДЕНО в кэше (обновили время использования)
Кэш (3/3): [ 1(4)  2(2)  3(3) ]

Запрос: 4 -> НЕ найдено, добавляем в кэш
Кэш (3/3): [ 1(4)  4(5)  3(3) ]
```

- Использовать два динамических массива одинакового размера:
 - `int *cache` - для хранения значений
 - `int *usage` - для хранения времени последнего использования

- Размер кэша должен задаваться при инициализации
- Функции:
 - `init_cache(int size)` - инициализация кэша заданного размера
 - `void get_value(int value)` - основной метод работы с кэшем
 - `void print_cache()` - вывод текущего состояния кэша
 - `void free_cache()` - освобождение памяти